

Conception et Implémentation d'une Intelligence Artificielle pour le Jeu d'Escampe

Rapport Final de Projet

Mathieu WAHARTE

Raphaël BINDA

Mai 2026

Résumé

Ce rapport détaille la conception et la mise en œuvre d'un agent autonome capable de jouer au jeu d'Escampe dans un environnement avec contraintes de temps réel. Nous y présentons nos choix d'implantation, notamment les structures de données optimisées (Bitboards), les algorithmes de recherche (Negamax avec élagage Alpha-Bêta, Iterative Deepening) ainsi que la fonction d'évaluation heuristique développée pour modéliser la complexité topologique du plateau. Nous abordons également les stratégies de placement initial, la gestion dynamique du temps d'exécution, et fournissons une analyse des performances validées par des tests rigoureux.

Table des matières

1	Introduction	2
2	Choix d'Implantation et Structures de Données	2
2.1	Représentation du Plateau (Bitboards)	2
2.2	Optimisations d'Implémentation	2
3	Algorithmes de Recherche du Meilleur Coup	2
3.1	Negamax et Élagage Alpha-Bêta	3
3.2	Approfondissement Itératif (Iterative Deepening)	3
3.3	Ordonnancement des Coups et Élagage Sélectif (Move Ordering & Pruning)	3
4	Fonction d'Évaluation (Heuristiques)	3
4.1	Heuristiques Implémentées et Testées	3
4.2	Intégration d'un Modèle d'Évaluation Profond : BandDPER	4
5	Stratégie Initiale : Placement et Ouvertures	4
5.1	Principes de Placement (Opening Book)	4
6	Gestion du Temps Réel	5

7	Analyse des Performances et Tests	5
7.1	Évaluation Elo et Test SPRT	6
7.2	Validation de la Conduite Autonome	6
8	Conclusion et Difficultés Rencontrées	6

1 Introduction

Dans le cadre de ce projet, l'objectif principal est de développer une Intelligence Artificielle capable de conduire de façon autonome une partie d'Écampe en temps limité (300s à 900s par joueur). L'agent doit communiquer avec un arbitre validant les coups et garantissant le respect des contraintes temporelles. Le jeu d'Écampe se caractérise par des règles de déplacement atypiques (dépendance à la bande ciblée par l'adversaire) et une asymétrie de la topologie du plateau. Ces spécificités requièrent des choix algorithmiques pointus, s'écartant parfois des implémentations classiques des jeux à information parfaite.

2 Choix d'Implantation et Structures de Données

Afin de garantir des performances optimales lors de l'exploration de l'arbre de jeu, les structures de données sous-jacentes doivent permettre une génération de coups et une évaluation d'états extrêmement rapides.

2.1 Représentation du Plateau (Bitboards)

Nous avons opté pour une représentation du plateau basée sur des *Bitboards* [2]. Le plateau de 6x6 cases est encodé sur des entiers de 64 bits (`long` en Java).

- Les opérations de manipulation de pièces (génération des coups, détection de collisions) se réduisent à des opérations bit à bit (ET, OU, décalages), exécutées en un seul cycle d'horloge.
- Cette approche permet d'éliminer les itérations coûteuses sur des tableaux multidimensionnels et minimise la création d'objets, limitant ainsi la pression sur le *Garbage Collector*.

2.2 Optimisations d'Implémentation

Plusieurs choix techniques ont été faits pour maximiser le nombre de nœuds explorés par seconde :

- Suppression des appels `instanceof` lors de la génération des coups via des méthodes dédiées par type de pièce.
- Utilisation de `System.nanoTime()` pour une résolution temporelle accrue sans coût de changement de contexte de l'OS.
- **[À COMPLÉTER]** Table de transposition via Zobrist Hashing [10] : utilisation d'une table de hachage plate (un tableau de `long`) pour éviter la réévaluation des positions déjà rencontrées, y compris la détection de répétition de position.

3 Algorithmes de Recherche du Meilleur Coup

La recherche du meilleur coup repose sur la théorie de la recherche adversarielle pour les jeux à somme nulle.

3.1 Negamax et Élagage Alpha-Bêta

Nous avons implémenté l'algorithme Negamax (une variante élégante du Minimax) couplé à un élagage Alpha-Bêta [1]. Cette approche unifie l'évaluation pour les deux joueurs, réduisant ainsi la complexité du code. L'élagage Alpha-Bêta permet de réduire drastiquement le facteur de branchement effectif de l'arbre de recherche en ignorant les branches dont il est prouvé qu'elles sont sous-optimales.

3.2 Approfondissement Itératif (Iterative Deepening)

Pour exploiter au mieux le temps imparti, la recherche est structurée autour d'un approfondissement itératif [3]. L'algorithme explore l'arbre progressivement : à profondeur 1, puis 2, puis 3, et ainsi de suite jusqu'à la limite `depthMax`. À chaque nouvelle itération de profondeur, le temps écoulé est vérifié via notre limite de calcul logicielle (*Soft Bound*). Si cette limite est atteinte en cours d'itération, l'algorithme abandonne l'itération incomplète en cours et renvoie immédiatement le meilleur coup validé lors de la *dernière itération complétée*. Ce mécanisme garantit qu'un coup fiable et profondément évalué est toujours disponible sans risque de dépassement.

[À COMPLÉTER] *Aspiration Windows* : utilisation de fenêtres de recherche réduites autour du score précédent pour maximiser les coupures d'élagage.

3.3 Ordonnancement des Coups et Élagage Sélectif (Move Ordering & Pruning)

% *TODO : Détailler les techniques d'ordonnancement implémentées.*

Un élagage Alpha-Bêta est maximal lorsque les meilleurs coups sont explorés en premier [6].

- **[À COMPLÉTER]** *Transposition Table Move* et *Killer Heuristic* [7] : essai prioritaire du meilleur coup de la profondeur précédente et des coups ayant provoqué des coupures ailleurs dans l'arbre.
- **[À COMPLÉTER]** *Null-move Pruning* [8] et *Late Move Reduction* [9] : élagage ou réduction de profondeur pour les branches jugées non prometteuses (futility pruning).

4 Fonction d'Évaluation (Heuristiques)

L'évaluation des feuilles de l'arbre de recherche se doit d'être précise tout en demeurant très rapide à calculer. Notre heuristique est une combinaison linéaire de plusieurs signaux topologiques.

4.1 Heuristiques Implémentées et Testées

La fonction d'évaluation s'articule autour des métriques suivantes :

1. **Distances d'Attaque et de Défense** : Calcul de la distance de Manhattan entre les Paladins et les Licornes adverses. Plus les attaquants sont proches, meilleur est le score. L'heuristique pénalise la proximité des Paladins adverses à notre Licorne. *Note : une implémentation avancée utilisant une distance en nombre de sauts (BFS ply) intégrant la contrainte des bandes a également été modélisée.*

2. **Échappatoire de la Licorne (Escapability)** : Compte direct du nombre de cases légales de fuite pour la Licorne de chaque joueur. Un adversaire avec peu d'échappatoires est considéré comme dominé positionnellement.
3. **Contrôle des Bandes (Band Control)** : La contrainte du jeu force l'adversaire à partir d'une bande spécifique. Le positionnement de nos Paladins sur des cases de bande 1 (où l'adversaire n'a aucun choix) est fortement valorisé.
4. **Pression Temporelle et Mobilité** : Récompense accordée pour un nombre de coups légaux plus élevé, et forte pénalité lorsqu'un joueur est forcé de passer son tour en raison du mécanisme de contrainte.

4.2 Intégration d'un Modèle d'Évaluation Profond : BandDPER

% TODO : Mettre ici un petit résumé de BandDPER si vous avez utilisé le réseau de neurones en move ordering, sinon mentionnez que ça a été exploré comme alternative.

Bien que l'heuristique manuelle soit performante, les contraintes spatiales fortement non linéaires d'Escampe (notamment la topologie des bandes) se prêtent bien à l'apprentissage. Le modèle **BandDPER** (Band-Aware Dual-Perspective Equivariant ResNet) a été conçu comme fonction d'évaluation alternative ou support à l'ordonnancement.

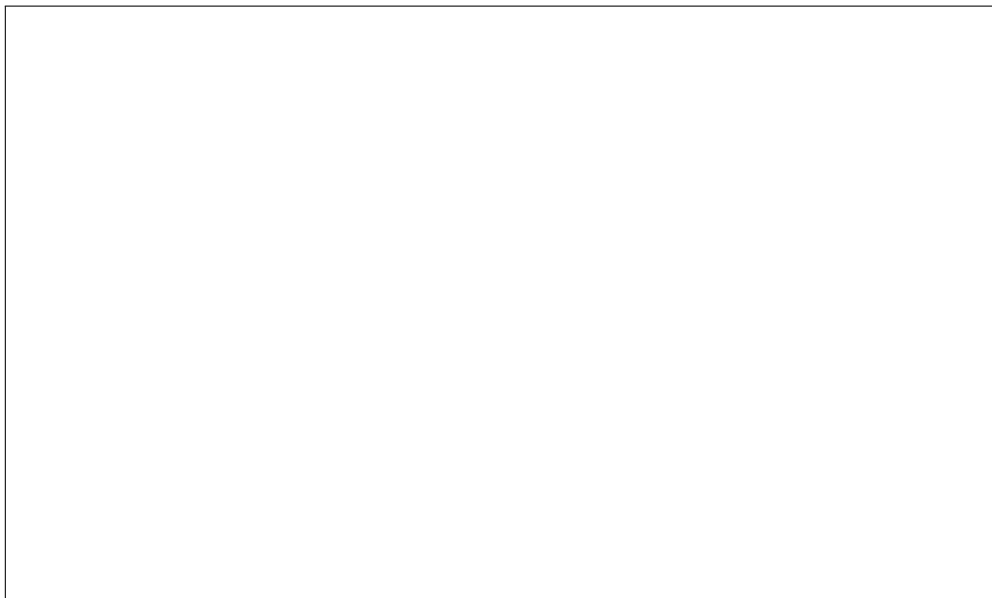


FIGURE 1 – Visualisation de l'architecture ou des courbes d'entraînement du modèle BandDPER.

5 Stratégie Initiale : Placement et Ouvertures

5.1 Principes de Placement (Opening Book)

L'issue de la partie dépend grandement du positionnement initial. Contrairement aux Échecs, le plateau de départ d'Escampe est défini par les joueurs. Notre agent intègre une bibliothèque de placements de départ (*Opening Book*) garantissant de solides propriétés structurelles :

- **Couverture de toutes les bandes** : Les Paladins doivent couvrir idéalement les bandes 1, 2 et 3. Un déséquilibre offrirait à l'adversaire l'opportunité de cibler une bande où l'agent serait contraint de passer.
- **Dispersion sur les colonnes** : Éviter les grappes de Paladins qui se bloquent mutuellement.
- **Sécurité de la Licorne** : Ne pas murer la Licorne avec ses propres pièces, tout en s'assurant qu'elle ne soit pas au centre géométrique sans couverture, privilégiant les cases "double-bande" pour l'équilibre entre mobilité et défense.

Différentes stratégies (*Balanced*, *Central Pressure*) ont été testées pour varier les parties et éviter la prévisibilité. Pendant le temps de calcul disponible au démarrage de l'application, l'agent utilise le *Pondering* [5] pour anticiper.

6 Gestion du Temps Réel

La communication avec le serveur impose des temps limites stricts sous peine de défaite immédiate. Nous avons implémenté un *TimeManager* [4] hybride avec gestion dynamique des bornes.

- **Estimation du temps par coup** : Le temps de base alloué est calculé en divisant le temps global restant par une estimation dynamique du nombre de coups restants. Cette estimation s'adapte selon la réserve : 40 coups supposés s'il reste beaucoup de temps ($>200s$), puis descend par paliers (25, 15) jusqu'à 8 coups en fin de partie critique.
- **Facteur de Complexité** : Un multiplicateur ajuste le budget temporel en fonction du nombre de coups légaux. Si le joueur n'a qu'un ou deux coups quasi-forcés (contrainte de bande), le budget est drastiquement réduit (facteur de 0.3) pour économiser du temps. Dans une position très ouverte (≥ 20 coups), le budget augmente (facteur 1.5).
- **Soft Bound / Hard Bound** :
 - La *Soft Bound* (limite logicielle) correspond au budget temporel de base ajusté. Une fois franchie, la boucle d'*Iterative Deepening* s'interrompt pour renvoyer le meilleur coup de la dernière profondeur complète.
 - La *Hard Bound* (limite matérielle) agit comme une sécurité absolue, fixée à $3\times$ la *Soft Bound* (avec un plafond strict à un cinquième du temps total restant). Si elle est atteinte en plein milieu de l'évaluation d'un nœud (lors des appels récursifs du Minimax), l'algorithme coupe instantanément la branche et retourne l'évaluation heuristique courante, empêchant ainsi le dépassement de temps fatal.

7 Analyse des Performances et Tests

Les performances de l'IA ont été évaluées via un moteur de test autonome opposant plusieurs versions de notre agent.

7.1 Évaluation Elo et Test SPRT

L'amélioration des heuristiques et l'optimisation des structures de données ont été quantifiées grâce au Sequential Probability Ratio Test (SPRT) [11] sur des milliers de parties générées automatiquement. Ce processus itératif garantit statistiquement qu'une modification logicielle apporte un gain Elo réel avant d'être validée.

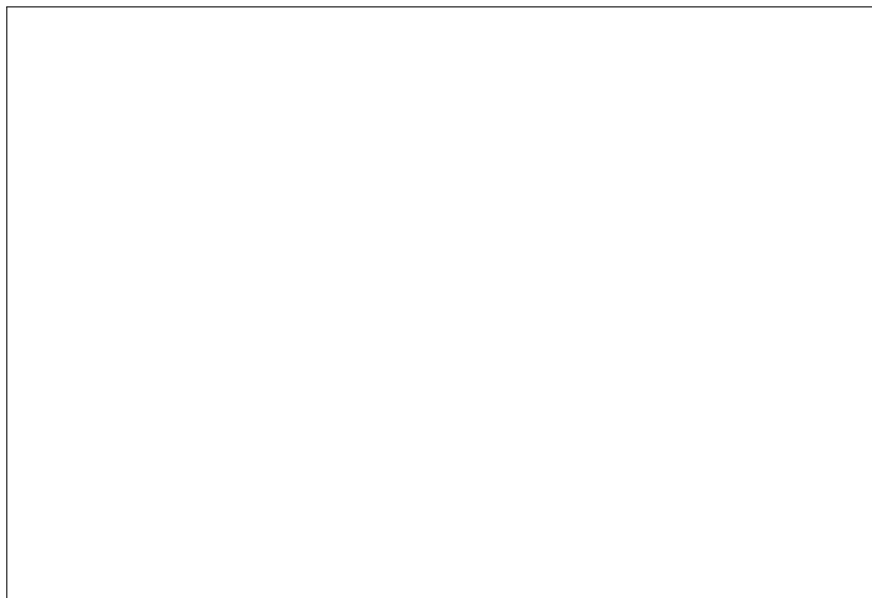


FIGURE 2 – Évolution du classement Elo des différentes configurations testées (tests SPRT).

7.2 Validation de la Conduite Autonome

Les protocoles de communication avec l'arbitre ont été validés localement en s'assurant qu'aucun dépassement de délai n'intervenait et que tous les coups générés restaient strictement conformes aux règles topologiques du jeu.

8 Conclusion et Difficultés Rencontrées

La mise au point de cette IA pour le jeu d'Escampe a été un défi algorithmique enrichissant. Les principales difficultés ont résidé dans :

- La compréhension initiale des symétries brisées du plateau (l'absence d'uniformité due aux bandes asymétriques complexifie fortement la pré-détermination de patterns géométriques réguliers).
- Le débogage subtil inhérent à l'algorithme Alpha-Bêta, notamment lors de l'intégration de l'ordonnancement dynamique des coups et de la gestion asynchrone des limites temporelles (*Hard bounds*).
- La conception d'une heuristique qui ne se laisse pas piéger par l'effet d'horizon, exacerbé dans Escampe par les réactions en chaîne de la mécanique de passe obligatoire.

L'agent final représente une synthèse robuste entre techniques classiques de la programmation de jeux à information parfaite et optimisation mathématique de topologie.

Références

- [1] Chess Programming Wiki, *Alpha-Beta*, <https://www.chessprogramming.org/Alpha-Beta>
- [2] Chess Programming Wiki, *Bitboards*, <https://www.chessprogramming.org/Bitboards>
- [3] Chess Programming Wiki, *Iterative Deepening*, https://www.chessprogramming.org/Iterative_Deepening
- [4] Chess Programming Wiki, *Time Management*, https://www.chessprogramming.org/Time_Management
- [5] Chess Programming Wiki, *Pondering*, <https://www.chessprogramming.org/Pondering>
- [6] Chess Programming Wiki, *Move Ordering*, https://www.chessprogramming.org/Move_Ordering
- [7] Chess Programming Wiki, *Killer Heuristic*, https://www.chessprogramming.org/Killer_Heuristic
- [8] Chess Programming Wiki, *Null Move Pruning*, https://www.chessprogramming.org/Null_Move_Pruning
- [9] Chess Programming Wiki, *Late Move Reductions*, https://www.chessprogramming.org/Late_Move_Reductions
- [10] Chess Programming Wiki, *Zobrist Hashing*, https://www.chessprogramming.org/Zobrist_Hashing
- [11] Chess Programming Wiki, *Match Statistics (SPRT)*, https://www.chessprogramming.org/Match_Statistics